

## Zaawansowane programowanie obiektowe

### laboratorium

|                         |                                                      |
|-------------------------|------------------------------------------------------|
| <b>Laboratorium nr:</b> | 3                                                    |
| <b>Tytuł:</b>           | IMPLEMENTACJA APLIKACJI BAZODANOWEJ PRZY UŻYCIU JDBC |
| <b>Student:</b>         | Szymon Banaszek, 1 grupa                             |
| <b>Nr legitymacji:</b>  | 124983                                               |
| <b>Prowadzący:</b>      | dr inż. Damian Szczegielniak                         |
| <b>Data:</b>            | Bydgoszcz, 03.06.2026                                |

## 1. Opis ćwiczenia

Ćwiczenie polegało na implementacji aplikacji bazodanowej w Javie z wykorzystaniem JDBC, wzorca DAO (Data Access Object) oraz graficznego interfejsu użytkownika w technologii JavaFX. Aplikacja zarządza projektami przechowywanymi w relacyjnej bazie danych HSQLDB, zapewniając pełną obsługę operacji CRUD (Create, Read, Update, Delete).

Użyte technologie i biblioteki:

- JavaFX 23.0.2 – graficzny interfejs użytkownika (TableView, Dialog, FXML, CSS)
- HSQLDB 2.7.4 – wbudowana relacyjna baza danych plikowa
- HikariCP 6.2.1 – pula połączeń z bazą danych (connection pool)
- SLF4J + Logback 1.5.17 – mechanizm rejestracji zdarzeń (logging)
- Gradle 9.3 – system budowania projektu z obsługą modułów Javy

## 2. Struktura projektu

Projekt zorganizowany jest w pakiety zgodnie z architekturą warstwową:

|                            |                                              |
|----------------------------|----------------------------------------------|
| 1. src/main/java/          |                                              |
| 2. com/project/model/      | - klasy modelu danych (Projekt)              |
| 3. com/project/datasource/ | - dostęp do bazy (DataSource, DbInitializer) |
| 4. com/project/dao/        | - warstwa DAO (ProjektDAO, ProjektDAOImpl)   |
| 5. com/project/controller/ | - kontroler JavaFX (ProjectController)       |
| 6. com/project/app/        | - punkt wejścia aplikacji                    |
| 7. module-info.java        | - deklaracja modułu Java                     |
| 8. src/main/resources/     |                                              |
| 9. fxml/ProjectFrame.fxml  | - layout interfejsu użytkownika              |
| 10. css/application.css    | - style tabeli                               |
| 11. logback.xml            | - konfiguracja rejestracji                   |

## 3. Warstwa modelu – klasa Projekt

Klasa Projekt odwzorowuje tabelę projekt w bazie danych. Zawiera trzy konstruktory: bezparametrowy, pełny oraz bez projectId (używany przy tworzeniu nowych rekordów), a także gettery i settery dla wszystkich pól.

```
1. package com.project.model;
2.
3. import java.time.LocalDate;
4. import java.time.LocalDateTime;
5.
6. public class Projekt {
7.     private Integer projectId;
8.     private String nazwa;
9.     private String opis;
10.    private LocalDateTime dataCzasUtworzenia;
11.    private LocalDate dataOddania;
12.
13.    public Projekt() {}
14.
15.    public Projekt(String nazwa, String opis, LocalDate dataOddania) {
16.        this.nazwa = nazwa; this.opis = opis; this.dataOddania = dataOddania;
17.    }
18.
19.    // gettery i settery (wygenerowane przez IDE)
20.    public Integer getProjektId() { return projectId; }
```

```

21.     public void setProjektId(Integer v) { this.projektId = v; }
22.     public String getNazwa() { return nazwa; }
23.     public void setNazwa(String v) { this.nazwa = v; }
24.     public String getOpis() { return opis; }
25.     public void setOpis(String v) { this.opis = v; }
26.     public LocalDateTime getDataCzasUtworzenia() { return dataCzasUtworzenia; }
27.     public void setDataCzasUtworzenia(LocalDateTime v) { this.dataCzasUtworzenia = v; }
28.     public LocalDate getDataOddania() { return dataOddania; }
29.     public void setDataOddania(LocalDate v) { this.dataOddania = v; }
30. }

```

## 4. Pula połączeń – klasa DataSource

Klasa DataSource implementuje wzorzec Singleton. Statyczny blok inicjalizujący tworzy dokładnie jedną instancję HikariDataSource przy pierwszym załadowaniu klasy. Prywatny konstruktor uniemożliwia tworzenie kolejnych instancji. Parametr sql.syntax\_pgs=true włącza obsługę typu SERIAL.

```

1. package com.project.datasources;
2.
3. import com.zaxxer.hikari.HikariConfig;
4. import com.zaxxer.hikari.HikariDataSource;
5. import java.sql.Connection;
6. import java.sql.SQLException;
7.
8. // Singleton - tylko jedna instancja HikariDataSource w całej aplikacji
9. public class DataSource {
10.     private static final String DB_URL =
11.         "jdbc:hsqldb:file:db/projekty;hsqldb.write_delay=false;sql.syntax_pgs=true";
12.     private static final HikariDataSource ds;
13.
14.     static { // Blok statyczny - inicjalizacja przy załadowaniu klasy
15.         HikariConfig config = new HikariConfig();
16.         config.setJdbcUrl(DB_URL);
17.         config.setUsername("admin");
18.         config.setPassword("admin");
19.         config.setMaximumPoolSize(1);
20.         ds = new HikariDataSource(config);
21.     }
22.
23.     private DataSource() {} // prywatny konstruktor - blokuje tworzenie instancji
24.
25.     public static Connection getConnection() throws SQLException {
26.         return ds.getConnection();
27.     }
28. }

```

## 5. Inicjalizacja bazy danych – klasa DbInitializer

Klasa DbInitializer tworzy schemat bazy danych przy starcie aplikacji. Wszystkie polecenia SQL wykonywane są w ramach jednej transakcji – dzięki temu albo wszystkie zostaną zastosowane, albo w przypadku błędu nastąpi rollback. Klauzula IF NOT EXISTS zapewnia idempotentność operacji.

```

1. public class DbInitializer {
2.     private static final String[] queries = {
3.         ""
4.         CREATE TABLE IF NOT EXISTS projekt(
5.             projekt_id SERIAL,
6.             nazwa VARCHAR(50) NOT NULL,

```

```

7.         opis VARCHAR(1000),
8.         dataczas_utworzenia TIMESTAMP DEFAULT now(),
9.         data_oddania DATE,
10.        CONSTRAINT projekt_pk PRIMARY KEY (projekt_id)
11.    );
12.    ...(analogicznie tabela zadanie)...
13.    """,
14.    """,
15.    CREATE INDEX IF NOT EXISTS projekt_nazwa_idx ON projekt(nazwa);
16.    ALTER TABLE zadanie ADD CONSTRAINT IF NOT EXISTS zadanie_projekt_fk
17.        FOREIGN KEY (projekt_id) REFERENCES projekt(projekt_id) ON DELETE CASCADE;
18.    """,
19.    };
20.
21.    public static void init() {
22.        try (Connection conn = DataSource.getConnection()) {
23.            conn.setAutoCommit(false);           // start transakcji
24.            try (Statement stmt = conn.createStatement()) {
25.                for (String q : queries) stmt.executeUpdate(q);
26.                conn.commit();                     // zatwierdź
27.            } catch (SQLException e) {
28.                conn.rollback();                   // cofnij przy błędzie
29.                throw new RuntimeException(e);
30.            }
31.        } catch (SQLException e) { throw new RuntimeException(e); }
32.    }
33. }

```

## 6. Wzorzec DAO – interfejs ProjektDAO

Interfejs ProjektDAO hermetyzuje konkretną implementację dostępu do danych. Dzięki temu pozostała część aplikacji jest niezależna od sposobu przechowywania danych – wystarczy dostarczyć nową implementację interfejsu.

```

1. public interface ProjektDAO {
2.     Projekt getProjekt(Integer projektId);
3.     void setProjekt(Projekt projekt);           // INSERT lub UPDATE
4.     void deleteProjekt(Integer projektId);
5.     List<Projekt> getProjekty(Integer offset, Integer limit);
6.     List<Projekt> getProjektyWhereNazwaLike(String nazwa, Integer offset, Integer limit);
7.     List<Projekt> getProjektyWhereDataOddaniaIs(LocalDate data, Integer offset, Integer limit);
8.     int getRowsNumber();
9.     int getRowsNumberWhereNazwaLike(String nazwa);
10.    int getRowsNumberWhereDataOddaniaIs(LocalDate dataOddania);
11. }

```

## 7. Implementacja DAO – klasa ProjektDAOImpl

### 7.1. Metoda setProjekt – INSERT i UPDATE

Metoda setProjekt realizuje zarówno wstawianie nowych rekordów (INSERT), jak i aktualizację istniejących (UPDATE). Decyzja podejmowana jest na podstawie wartości pola projektId – jeśli wynosi null, wykonywany jest INSERT i po operacji pobierany jest wygenerowany klucz główny.

```

1. @Override
2. public void setProjekt(Projekt projekt) {
3.     boolean isInsert = projekt.getProjektId() == null;
4.     String query = isInsert
5.         ? "INSERT INTO projekt(nazwa, opis, dataczas_utworzenia, data_oddania) VALUES (?, ?, ?, ?)"

```

```

6.      : "UPDATE projekt SET nazwa=?, opis=?, dataczas_utworzenia=?, data_oddania=?"
7.      + " WHERE projekt_id=?";
8.      try (Connection conn = DataSource.getConnection();
9.          PreparedStatement ps = conn.prepareStatement(query, RETURN_GENERATED_KEYS)) {
10.         ps.setString(1, projekt.getNazwa());
11.         ps.setString(2, projekt.getOpis());
12.         if (projekt.getDataCzasUtworzenia() == null)
13.             projekt.setDataCzasUtworzenia(LocalDateTime.now());
14.         ps.setObject(3, projekt.getDataCzasUtworzenia());
15.         ps.setObject(4, projekt.getDataOddania());
16.         if (!isInsert) ps.setInt(5, projekt.getProjektId());
17.         int rows = ps.executeUpdate();
18.         if (isInsert && rows > 0) { // pobierz wygenerowany klucz
19.             ResultSet keys = ps.getGeneratedKeys();
20.             if (keys.next()) projekt.setProjektId(keys.getInt(1));
21.         }
22.     } catch (SQLException e) { throw new RuntimeException(e); }
23. }

```

## 7.2. Metoda getProjekty – paginacja

Metoda `getProjekty` pobiera stronę wyników z obsługą opcjonalnego `OFFSET` i `LIMIT`. Budowanie zapytania SQL jest dynamiczne – klauzule dodawane są tylko gdy parametr nie jest null.

```

1. @Override
2. public List<Projekt> getProjekty(Integer offset, Integer limit) {
3.     List<Projekt> lista = new ArrayList<>();
4.     String query = "SELECT * FROM projekt ORDER BY dataczas_utworzenia DESC"
5.         + (offset != null ? " OFFSET ?" : "")
6.         + (limit != null ? " LIMIT ?" : "");
7.     try (Connection conn = DataSource.getConnection();
8.         PreparedStatement ps = conn.prepareStatement(query)) {
9.         int i = 1;
10.        if (offset != null) ps.setInt(i++, offset);
11.        if (limit != null) ps.setInt(i, limit);
12.        try (ResultSet rs = ps.executeQuery()) {
13.            while (rs.next()) {
14.                Projekt p = new Projekt();
15.                p.setProjektId(rs.getInt("projekt_id"));
16.                p.setNazwa(rs.getString("nazwa"));
17.                p.setOpis(rs.getString("opis"));
18.                p.setDataCzasUtworzenia(rs.getObject("dataczas_utworzenia",
19.                    LocalDateTime.class));
20.                p.setDataOddania(rs.getObject("data_oddania", LocalDate.class));
21.                lista.add(p);
22.            }
23.        } catch (SQLException e) { throw new RuntimeException(e); }
24.        return lista;
25.    }

```

Pozostałe metody (`getProjekt`, `deleteProjekt`, `getProjektyWhereNazwaLike`, `getProjektyWhereDataOddanials`, `getRowsNumber` oraz warianty liczące wiersze) zostały zaimplementowane analogicznie, z użyciem `PreparedStatement` i `try-with-resources`.

## 8. Kontroler JavaFX – ProjectController

### 8.1. Inicjalizacja – metoda initialize()

Metoda initialize() wywoływana jest automatycznie przez JavaFX po wstrzyknięciu wszystkich komponentów zadeklarowanych adnotacją @FXML. Inicjalizuje kolumny tabeli, formatowanie dat, kolumnę z przyciskami akcji oraz uruchamia pierwsze pobieranie danych w tle.

```

1. @FXML
2. public void initialize() {
3.     search4 = ""; pageNo = 0; pageSize = 10;
4.     cbPageSizes.getItems().addAll(5, 10, 20, 50, 100);
5.     cbPageSizes.setValue(pageSize);
6.
7.     colId.setCellValueFactory(new PropertyValueFactory<>("projektId"));
8.     colNazwa.setCellValueFactory(new PropertyValueFactory<>("nazwa"));
9.     colOpis.setCellValueFactory(new PropertyValueFactory<>("opis"));
10.    colDataCzasUtworzenia.setCellValueFactory(new PropertyValueFactory<>("dataCzasUtworzenia"));
11.    colDataOddania.setCellValueFactory(new PropertyValueFactory<>("dataOddania"));
12.
13.    // Formatowanie LocalDateTime w kolumnie
14.    colDataCzasUtworzenia.setCellFactory(col -> new TableCell<>() {
15.        protected void updateItem(LocalDateTime item, boolean empty) {
16.            super.updateItem(item, empty);
17.            setText(item == null || empty ? null : dateTimeFormatter.format(item));
18.        }
19.    });
20.
21.    // Kolumna z przyciskami (Zadania / Edytuj / Usuń)
22.    TableColumn<Projekt, Void> colEdit = new TableColumn<>("Edycja");
23.    colEdit.setCellFactory(col -> new TableCell<>() {
24.        Button btnTask = new Button("Zadania");
25.        Button btnEdit = new Button("Edytuj");
26.        Button btnRemove = new Button("Usuń");
27.        // ... konfiguracja i rejestracja zdarzeń ...
28.    });
29.
30.    projekty = FXCollections.observableArrayList();
31.    tblProjekt.setItems(projekty);
32.    wykonawca.execute(() -> loadPage(search4, pageNo, pageSize));
33. }

```

### 8.2. Ładowanie strony – metoda loadPage()

Metoda loadPage() wykonywana jest w osobnym wątku (ExecutorService) – dzięki temu interfejs nie jest blokowany podczas pobierania danych z bazy. Obsługuje trzy tryby wyszukiwania rozróżniane wyrażeniami regularnymi: po identyfikatorze, po dacie i po nazwie. Wyniki przekazywane są do wątku JavaFX przez Platform.runLater().

```

1. private void loadPage(String search4, Integer pageNo, Integer pageSize) {
2.     try {
3.         List<Projekt> list = new ArrayList<>();
4.         if (search4 != null && !search4.isEmpty()) {
5.             if (search4.matches("[0-9]+")) {
6.                 // wyszukiwanie po identyfikatorze
7.                 Projekt p = projektDAO.getProjekt(Integer.parseInt(search4));
8.                 if (p != null) list.add(p);
9.             } else if (search4.matches("[0-9]{4}-(0[1-9]|1[0-2])-(0[1-9]|1[2][0-9]|3[01])$")) {
10.                // wyszukiwanie po dacie oddania
11.                list.addAll(projektDAO.getProjektyWhereDataOddaniaIs(
12.                    LocalDate.parse(search4, dateFormatter), pageNo*pageSize, pageSize));
13.            } else {
14.                // wyszukiwanie po nazwie (LIKE)

```

```

15.         list.addAll(projektDAO.getProjektyWhereNazwaLike(search4, pageNo*pageSize,
16.         });
17.     } else {
18.         list.addAll(projektDAO.getProjekty(pageNo * pageSize, pageSize));
19.     }
20.     Platform.runLater(() -> { projekty.clear(); projekty.addAll(list); });
21. } catch (RuntimeException e) {
22.     Platform.runLater(() -> showError("Bład pobierania danych", e.getMessage()));
23. }
24. }

```

### 8.3. Puła wątków i zamykanie aplikacji

Kontroler zarządza pulą jednego wątku roboczego (`Executors.newFixedThreadPool(1)`). Metoda `shutdown()` wywoływana jest przy zamknięciu okna aplikacji, zapewniając poprawne zakończenie wszystkich zadań z kolejki przed zatrzymaniem programu.

| Id | Nazwa          | Opis    | Data utworzenia     | Data oddania | Edycja                    |
|----|----------------|---------|---------------------|--------------|---------------------------|
| 1  | Testowo dodane | tseadas | 2026-06-03 14:00:46 | 2026-06-10   | Zadania<br>Edytuj<br>Usuń |
| 2  | fsad           | sdaa    | 2026-06-03 14:01:41 |              | Zadania<br>Edytuj<br>Usuń |
| 3  | dsada          | dasdaw  | 2026-06-03 14:10:39 | 2026-06-20   | Zadania<br>Edytuj<br>Usuń |
|    |                |         |                     |              |                           |
|    |                |         |                     |              |                           |
|    |                |         |                     |              |                           |
|    |                |         |                     |              |                           |
|    |                |         |                     |              |                           |
|    |                |         |                     |              |                           |

### 8.4. Edycja i tworzenie projektu – metoda `edytujProjekt()`

Metoda `edytujProjekt()` wyświetla okno dialogowe (`Dialog<Projekt>`) z formularzem edycji. Jeśli `projektId` jest null, wyświetlany jest napis 'Dodawanie projektu', w przeciwnym razie 'Edycja danych projektu'. Zapis odbywa się asynchronicznie w wątku `ExecutorService`.

Dodawanie projektu

Id:

Data utw.:

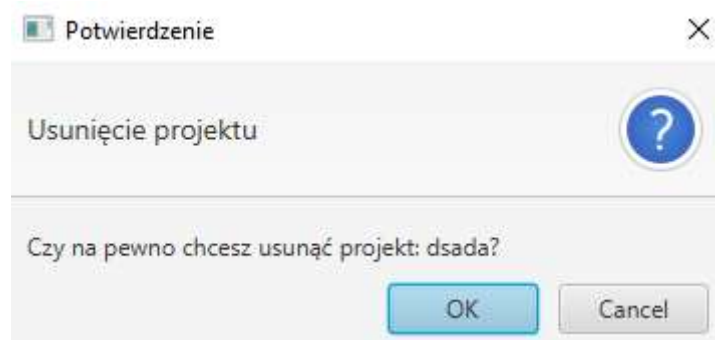
Nazwa:

Opis:

Data oddania:

### 8.5. Usuwanie projektu – metoda `usunProjekt()`

Przed usunięciem wyświetlane jest okno potwierdzenia (`Alert.CONFIRMATION`). Po zatwierdzeniu operacja wykonywana jest asynchronicznie, a wpis usuwany jest z listy `ObservableList` bez ponownego ładowania strony.



## 9. Klasa główna i moduł – `ProjectClientApplication` i `module-info.java`

Klasa `ProjectClientApplication` rozszerza `javafx.application.Application`. W metodzie `main()` inicjalizowany jest schemat bazy (`DbInitializer.init()`), a następnie uruchamiana aplikacja JavaFX. Metoda `start()` używa `setControllerFactory` do przekazania obiektu `ProjektDAO` do kontrolera, a `setOnCloseRequest` rejestruje wywołanie `controller.shutdown()` przy zamknięciu okna.

### 9.1. Plik `module-info.java`

Plik `module-info.java` definiuje moduł projektu, eksportowane pakiety oraz wymagane zależności. Dyrektywy `opens` są wymagane przez JavaFX do działania mechanizmu wstrzykiwania (`FXML injection`) przez refleksję.

```

1. module project.jfx.client {
2.     exports com.project.datasource;
3.     exports com.project.dao;
4.     exports com.project.model;
5.     exports com.project.app;

```



```
6.     exports com.project.controller;
7.
8.     requires javafx.base;
9.     requires javafx.fxml;
10.    requires javafx.controls;
11.    requires transitive javafx.graphics;
12.    requires com.zaxxer.hikari;
13.    requires transitive java.sql;
14.    requires org.hsqldb;
15.    requires org.slf4j;
16.    requires ch.qos.logback.classic;
17.    requires ch.qos.logback.core;
18.
19.    opens com.project.app          to javafx.graphics, javafx.fxml, javafx.base, javafx.controls;
20.    opens com.project.model       to javafx.graphics, javafx.fxml, javafx.base, javafx.controls;
21.    opens com.project.controller to javafx.graphics, javafx.fxml, javafx.base, javafx.controls;
22. }
```

## 10. Wnioski

Wzorzec DAO skutecznie oddziela logikę dostępu do danych od warstwy prezentacji – zmiana bazy danych wymaga wyłącznie nowej implementacji interfejsu ProjektDAO, bez modyfikowania GUI.

Użycie HikariCP z MaximumPoolSize=1 dla bazy plikowej HSQLDB jest celowe – baza nie obsługuje współbieżnego zapisu z wielu połączeń, a pula jednego wątku w ExecutorService eliminuje wyścigi.

Platform.runLater() jest kluczowe w architekturze JavaFX – wszelkie modyfikacje komponentów GUI muszą być wykonywane w wątku JavaFX Application Thread, nie w wątkach roboczych.

Mechanizm logowania (SLF4J + Logback) zastępuje System.out.println() – umożliwia zapis logów do pliku z automatyczną rotacją i filtrowanie po poziomie (INFO, WARN, ERROR).